Computation (and Communication) Efficient Second-order Methods

Chengchang Liu (CUHK)

June 16, 2023

Outline

1. Why Second-Order Methods?
2. Computation Efficient Methods for Single Agent Optimization
   - SR-k: Bridging the quasi-Newton and the Newton
   - SQN: From convex optimization to more
   - PQN: Faster convergence by exploiting special structure
3. Comunication Efficient Methods for Distributed Optimization
4. Conclusion

Outline

1. Why Second-Order Methods?
2. Computation Efficient Methods for Single Agent Optimization
   - SR-k: Bridging the quasi-Newton and the Newton
   - SQN: From convex optimization to more
   - PQN: Faster convergence by exploiting special structure
3. Comunication Efficient Methods for Distributed Optimization
4. Conclusion

Motivation

Let's start with the minimization problems

$$\min_{\mathbf{x}\in\mathbb{R}^d} f(\mathbf{x}). \tag{1}$$

Definition (Smoothness)

We say $f(\mathbf{x})$ is p-th order L_p-Lipschitz continuous if

$$\|\nabla^p f(\mathbf{x}) - \nabla^p f(\mathbf{y})\| \leq L_p \|\mathbf{x} - \mathbf{y}\|.$$

Definition (Convexity)

We say $f(\mathbf{x})$ is μ-strongly convex if

$$\nabla^2 f(\mathbf{x}) \succeq \mu \mathbf{I},$$

and we further define the condition number $\kappa \stackrel{\text{def}}{=} L_1/\mu \ (\gg 1)$

Motivation

We define the oracle of p-th order method

$$T_{p,L_p}(\mathbf{x}) := \arg\min_{\mathbf{y}} f(\mathbf{x}) + \sum_{i=1}^{p} \frac{1}{i!} \nabla^i f(\mathbf{x})[\mathbf{y}-\mathbf{x}]^i + \frac{L_p}{(p+1)!} \|\mathbf{y}-\mathbf{x}\|^{p+1}.$$

- First-order method:

$$\mathbf{x}_{t+1} = T_{1,L_1}(\mathbf{x}_t) = \mathbf{x}_t - \frac{1}{L_1} \nabla f(\mathbf{x}_t)$$

- Second-order method:

$$\mathbf{x}_{t+1} = T_{2,L_2}(\mathbf{x}_t) = \mathbf{x}_t - \left(\nabla^2 f(\mathbf{x}_t) + L_2 r_t \mathbf{I}_d\right)^{-1} \nabla f(\mathbf{x}_t)$$

Motivation

Comparison between first and second-order methods

Methods	Nonconvex	Strongly-convex
Gradient descent	$\mathcal{O}(\epsilon^{-2})$	linear
(Cubic-regularized) Newton	$\mathcal{O}(\epsilon^{-3/2})$	superlinear

- No variants of first-order methods can achieve the rates of second-order methods
- Second-order methods perform very well on small-medium scale models

Motivation (drawback of second-order method)

- For single agent problems

$$\min_{\mathbf{x}\in\mathbb{R}^d} f(\mathbf{x}),$$

  The computation cost of calculating the Hessian and its inverse is too high $\mathcal{O}(d^3)$.

- For multi-agent problems (distributed optimization)

$$\min_{\mathbf{x}\in\mathbb{R}^d} f(\mathbf{x}) = \frac{1}{m}\sum_{i=1}^{m} f_i(\mathbf{x}),$$

  The communication cost of Hessian between the clients and the server is too high $\mathcal{O}(d^2)$.

Motivation

Can we design second-order methods which enjoy both the superiority of the convergence and the computation (communication) efficiency?

Outline

1. Why Second-Order Methods?
2. Computation Efficient Methods for Single Agent Optimization
   - SR-k: Bridging the quasi-Newton and the Newton
   - SQN: From convex optimization to more
   - PQN: Faster convergence by exploiting special structure
3. Comunication Efficient Methods for Distributed Optimization
4. Conclusion

Quasi-Newton methods

Consider the single-agent minimization problems

$$\min_{\mathbf{x}} f(\mathbf{x}) \triangleq \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x}), \tag{2}$$

where f is μ-strongly convex.
Quasi-Newton methods enjoy both superlinear convergence rate and computation efficiency

Order	flops per iteration	Convergence
first-order	$\mathcal{O}(d^2)$ (nd)	linear $(\lim_{t\to\infty} \lambda_{t+1}/\lambda_t < 1)$
second-order	$\mathcal{O}(d^3)$ $(nd^2 + d^3)$	superlinear $(\lambda_{t+1}/\lambda_t = \lambda_t)$
Quasi-Newton	$\mathcal{O}(d^2)$ $(nd + d^2)$	superlinear $(\lim_{t\to\infty} \lambda_{t+1}/\lambda_t = 0)$

Quasi-Newton methods

Consider the iteration of quasi-Newton

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1}\nabla f(\mathbf{x}_t) \\ \mathbf{G}_{t+1} = \mathbf{G}_t + \underbrace{\Delta_t}_{\text{rank}=1,2} = \underbrace{\text{Update}}_{\text{SR1/BFGS/DFP}} (\underbrace{\mathbf{G}_t}_{\succ \mathbf{0}}, \underbrace{\mathbf{H}_{t+1}}_{\succ \mathbf{0}}, \mathbf{u}_t), \end{cases}$$

where $\mathbf{u}_t \in \mathbb{R}^{d\times 1}$, which satisfies

$$\mathbf{G}_{t+1}\mathbf{u}_t = \mathbf{H}_{t+1}\mathbf{u}_t.$$

Take BFGS udpate for example:

$$\text{BFGS}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{u}_t) \triangleq \mathbf{G}_t - \frac{\mathbf{G}_t\mathbf{u}_t\mathbf{u}_t^\top\mathbf{G}_t}{\mathbf{u}_t^\top\mathbf{G}_t\mathbf{u}_t} + \frac{\mathbf{H}_{t+1}\mathbf{u}_t\mathbf{u}_t^\top\mathbf{H}_{t+1}}{\mathbf{u}_t^\top\mathbf{H}_{t+1}\mathbf{u}_t}$$

Quasi-Newton methods

$$\mathrm{BFGS}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{u}_t) \triangleq \mathbf{G}_t - \frac{\mathbf{G}_t \mathbf{u}_t \mathbf{u}_t^\top \mathbf{G}_t}{\mathbf{u}_t^\top \mathbf{G}_t \mathbf{u}_t} + \frac{\mathbf{H}_{t+1} \mathbf{u}_t \mathbf{u}_t^\top \mathbf{H}_{t+1}}{\mathbf{u}_t^\top \mathbf{H}_{t+1} \mathbf{u}_t}$$

The classical quasi-Newton methods (1970s):

- We implement update by

$$\mathbf{u}_t = \mathbf{x}_{t+1} - \mathbf{x}_t \qquad \text{and} \qquad \mathbf{H}_{t+1} = \int_0^1 \nabla^2 f(\mathbf{x}_t + s\mathbf{u}_t)\,\mathrm{d}s,$$

  which leads to $\underbrace{\mathbf{G}_{t+1}\mathbf{u}_t = \mathbf{H}_{t+1}\mathbf{u}_t = \nabla f(\mathbf{x}_{t+1}) - \nabla f(\mathbf{x}_t) \triangleq \mathbf{v}_t}_{\text{secant equation}}$.

- For example, the classical BFGS can be written as

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1} \nabla f(\mathbf{x}_t) \\ \mathbf{G}_{t+1} = \mathrm{BFGS}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{u}_t) = \mathbf{G}_t - \dfrac{\mathbf{G}_t \mathbf{u}_t \mathbf{u}_t^\top \mathbf{G}_t}{\mathbf{u}_t^\top \mathbf{G}_t \mathbf{u}_t} + \dfrac{\mathbf{v}_t \mathbf{v}_t^\top}{\mathbf{v}_t^\top \mathbf{u}_t}. \end{cases}$$

Quasi-Newton methods

Greedy and Random Quasi-Newton Methods (2010-2020):

- 1. greedy quasi-Newton [Ronomanov and Nesterov 2020]

$$\mathbf{u}_t = \underset{\mathbf{u}\in\{\mathbf{e}_1,\ldots,\mathbf{e}_d\}}{\arg\max} \frac{\mathbf{u}^\top \mathbf{G}_t \mathbf{u}}{\mathbf{u}^\top \mathbf{H}_{t+1}\mathbf{u}}, \qquad \mathbf{H}_{t+1} = \nabla^2 f(\mathbf{x}_{t+1})$$

  2. Randomized quasi-Newton [Gower and Richtárik 2016], [Lin et.al 2021]

$$\mathbf{u}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad \mathbf{H}_{t+1} = \nabla^2 f(\mathbf{x}_{t+1}).$$

  They satisfies $\mathbf{G}_{t+1}\mathbf{u}_t = \mathbf{H}_{t+1}\mathbf{u}_t = \underbrace{\nabla^2 f(\mathbf{x}_{t+1})\mathbf{u}_t}_{\text{Hessian-Vector Product}}$.

- For example, the greedy/random BFGS can be written as

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1}\nabla f(\mathbf{x}_t) \\ \tilde{\mathbf{G}}_t = (1 + M\|\mathbf{x}_{t+1} - \mathbf{x}_t\|_{\mathbf{x}_t})\mathbf{G}_t \\ \mathbf{G}_{t+1} = \tilde{\mathbf{G}}_t - \dfrac{\tilde{\mathbf{G}}_t \mathbf{u}_t \mathbf{u}_t^\top \tilde{\mathbf{G}}_t}{\mathbf{u}_t^\top \tilde{\mathbf{G}}_t \mathbf{u}_t} + \dfrac{\nabla^2 f(\mathbf{x}_{t+1})\mathbf{u}_t\mathbf{u}_t^\top \nabla^2 f(\mathbf{x}_{t+1})}{\mathbf{u}_t^\top \nabla^2 f(\mathbf{x}_{t+1})\mathbf{u}_t}. \end{cases}$$

Quasi-Newton methods

Block Randomized quasi-Newton [R.B.Schnabel 1980], [Gower and Richtárik 2016]

- BFGS update, choose $\mathbf{u}_t \in \mathbb{R}^{d\times 1}$

$$\mathrm{BFGS}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{u}_t) \triangleq \mathbf{G}_t - \frac{\mathbf{G}_t\mathbf{u}_t\mathbf{u}_t^\top\mathbf{G}_t}{\mathbf{u}_t^\top\mathbf{G}_t\mathbf{u}_t} + \frac{\mathbf{H}_{t+1}\mathbf{u}_t\mathbf{u}_t^\top\mathbf{H}_{t+1}}{\mathbf{u}_t^\top\mathbf{H}_{t+1}\mathbf{u}_t} \tag{3}$$

- Block BFGS update, choose $\mathbf{U}_t \in \mathbb{R}^{d\times k}$, which generalize equation (3)

$$\mathrm{BlockBFGS}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{U}_t) \triangleq$$
$$\mathbf{G}_t - \mathbf{G}_t\mathbf{U}_t\left(\mathbf{U}_t^\top\mathbf{G}_t\mathbf{U}_t\right)^{-1}\mathbf{U}_t^\top\mathbf{G}_t + \mathbf{H}_{t+1}\mathbf{U}_t\left(\mathbf{U}_t^\top\mathbf{H}_{t+1}\mathbf{U}_t\right)^{-1}\mathbf{U}_t^\top\mathbf{H}_{t+1}.$$

Choosing $\mathbf{U}_t \sim \mathcal{D}$ as a random matrix leads to the randomized block BFGS methods.

Quasi-Newton methods

Although quasi-Newton methods is well-studied, there are lots of problems remain to solve.

- For convex optimization, block quasi-Newton methods with rank-k update are better in practice, but current theory cannot illustrate this.
- The quasi-Newton update are developed for convex optimization, where the object matrix is positive definite $\nabla^2 f(\mathbf{x}) \succ \mathbf{0}$.
  1. In minimax optimization, the Hessian is indefinite.
  2. In general non-linear equation , the Jacobian is even non-square!

Quasi-Newton methods

The following papers leverage the above problems.

- **Chengchang Liu**, Cheng Chen and Luo Luo. Symmetric Rank-k Methods. *arXiv Preprint*: 2303.16188.
- **Chengchang Liu** and Luo Luo. Quasi-Newton methods for saddle point problems. *NeurIPS*, 2022. *(Spotlight)*
- **Chengchang Liu**, Shuxian Bi, Luo Luo and John C.S. Lui. Partial-quasi-Newton methods: Efficient algorithms for minimax optimization problems with unbalanced dimensionality. *KDD*, 2022. *(Best Paper Award Runner-Up)*

SR-k

Consider the general form of update for convex optimization

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1} \nabla f(\mathbf{x}_t) \\ \mathbf{G}_{t+1} = \mathrm{Update}(\mathbf{G}_t), \end{cases}$$

- Gradient Descent: $\mathbf{G}_{t+1} = \mathbf{G}_t = L\mathbf{I} \rightarrow$ rank-0 update
- Newton Method: $\mathbf{G}_{t+1} = \nabla^2 f(\mathbf{x}_{t+1}) \rightarrow$ rank-d update
- Quasi-Newton Method: $\mathbf{G}_{t+1} = \mathbf{G}_t + \Delta_t$, where $\mathrm{rank}(\Delta_t) = 1, 2$

	Gradient-Descent	SR1/BFGS	Newton Method
λ_{t+1}/λ_t	$(1 - 1/\kappa)$	$\mathcal{O}\left((1 - 1/d)^t\right)$	$\mathcal{O}(\lambda_t)$
Rank	0	1 or 2	d

SR-k

What if we have rank-k update on $\mathbf{G}_t$ at each iteration? ($k \in [d]$)

	GD	SR1/BFGS	?	Newton
λ_{t+1}/λ_t	$\left(1-\frac{1}{\kappa}\right)$	$\mathcal{O}\left((1-1/d)^t\right)$	?	$\mathcal{O}(\lambda_t)$
Rank	0	1 or 2	$k \in [d]$	d

SR-k

We proposed symmetric rank-k method!

	GD	SR1/BFGS	SR-k	Newton
λ_{t+1}/λ_t	$\left(1-\frac{1}{\kappa}\right)$	$\mathcal{O}\big((1-1/d)^t\big)$	$\begin{cases}\mathcal{O}\big((1-k/d)^t\big), & k\in[d-1]\\ \mathcal{O}\big(\lambda_t\big), & k=d\end{cases}$	$\mathcal{O}(\lambda_t)$
Rank	0	1 or 2	$k\in[d]$	d

SR-k: matrix approximation

We first propose SR-k update strategy to approximate $\mathbf{A} \succ \mathbf{0}$.

$$\text{SR-}k(\mathbf{G}, \mathbf{A}, \mathbf{U}) \triangleq \mathbf{G} - (\mathbf{G} - \mathbf{A})\mathbf{U}(\mathbf{U}^\top(\mathbf{G} - \mathbf{A})\mathbf{U})^{-1}\mathbf{U}^\top(\mathbf{G} - \mathbf{A}).$$

- $\mathbf{AU} = [\mathbf{Au}_1, \cdots, \mathbf{Au}_k]$, calculation of $\mathbf{Au}_i$ can be conducted in parallel.
- $\mathbf{U} \in \mathbb{R}^{d\times k}$ is some full rank matrix which can be conducted efficiently
  1. For randomized strategy, we sample each entry of $\mathbf{U}$ according to $\mathcal{N}(0, 1)$ independently.
  2. For greedy strategy, We choose $\mathbf{U} = [\mathbf{e}_{i_1}; \cdots; \mathbf{e}_{i_k}] \in \mathbb{R}^{d\times k}$, $i_1, \ldots, i_k$ are the indices for the largest k entries in the diagonal of $\mathbf{G} - \mathbf{A}$.

SR-k: matrix approximation

Convergence behaviour of SR-k update.

$$\text{SR-}k(\mathbf{G}, \mathbf{A}, \mathbf{U}) \triangleq \mathbf{G} - (\mathbf{G} - \mathbf{A})\mathbf{U}(\mathbf{U}^\top(\mathbf{G} - \mathbf{A})\mathbf{U})^{-1}\mathbf{U}^\top(\mathbf{G} - \mathbf{A}).$$

Lemma (Linear convergence for matrix approximation)

$\mathbf{G}_+ = \mathrm{SR}\text{-}k(\mathbf{G}, \mathbf{A}, \mathbf{U})$ *and* $\tau_{\mathbf{A}}(\mathbf{G}) = \mathrm{tr}(\mathbf{G} - \mathbf{A})$, *then*

$$\mathbb{E}\left[\tau_{\mathbf{A}}(\mathbf{G}_+)\right] \leq \left(1 - \frac{k}{d}\right)\tau_{\mathbf{A}}(\mathbf{G}).$$

SR-k: convex optimization

The SR-k methods use the following update scheme [Rodomanov and Nesterov, 2022]

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1}\nabla f(\mathbf{x}_t) \\ \tilde{\mathbf{G}}_t = (1 + M\|\mathbf{x}_{t+1} - \mathbf{x}_t\|_{\mathbf{x}_t})\mathbf{G}_t \\ \mathbf{G}_{t+1} = \text{SR-}k(\tilde{\mathbf{G}}_t, \nabla^2 f(\mathbf{x}_{t+1}), \mathbf{U}_t). \end{cases}$$

Theorem

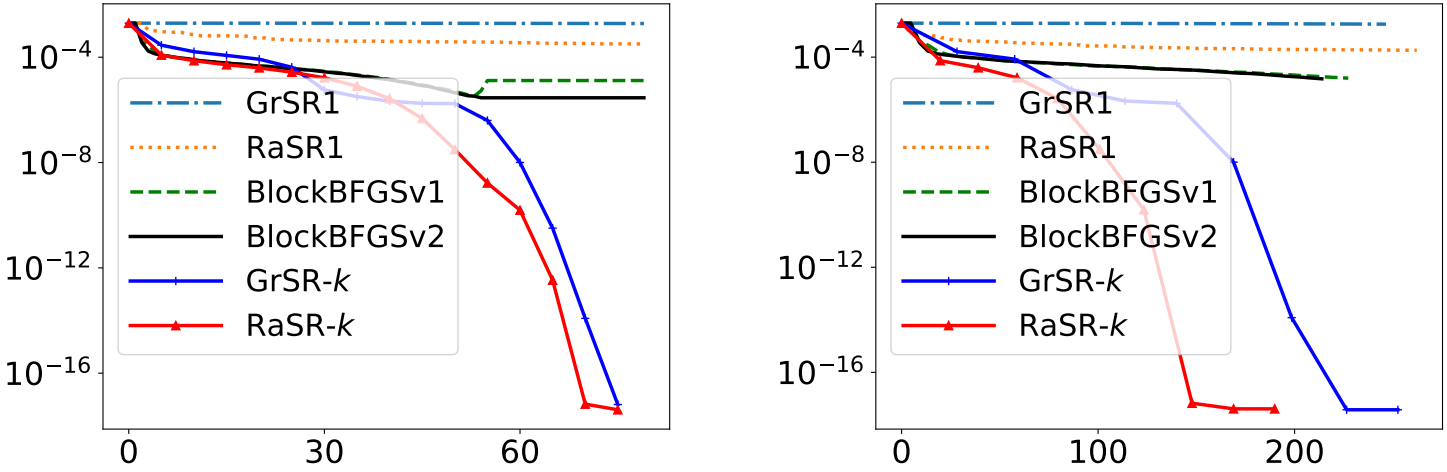
Using SR-k *to minimize the object function which is* μ*-strongly convex and its first and second order derivatives are Lipschitz continuous, we have the superlinear rate of*

$$\mathbf{E}\left[\lambda_{t+1}/\lambda_t\right] = \begin{cases} \mathcal{O}\big((1-k/d)^t\big), & k \in [d-1] \\ \mathcal{O}(\lambda_t), & k = d \end{cases} \tag{4}$$

SR-k: Experiments on logistic regression

$$f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} \ln\left(1 + \exp\left(-b_i \mathbf{x}^\top \mathbf{a}_i\right)\right) + \lambda \|\mathbf{x}\|_2^2,$$

Convergence behavior of SR-k with other quasi-Newton methods

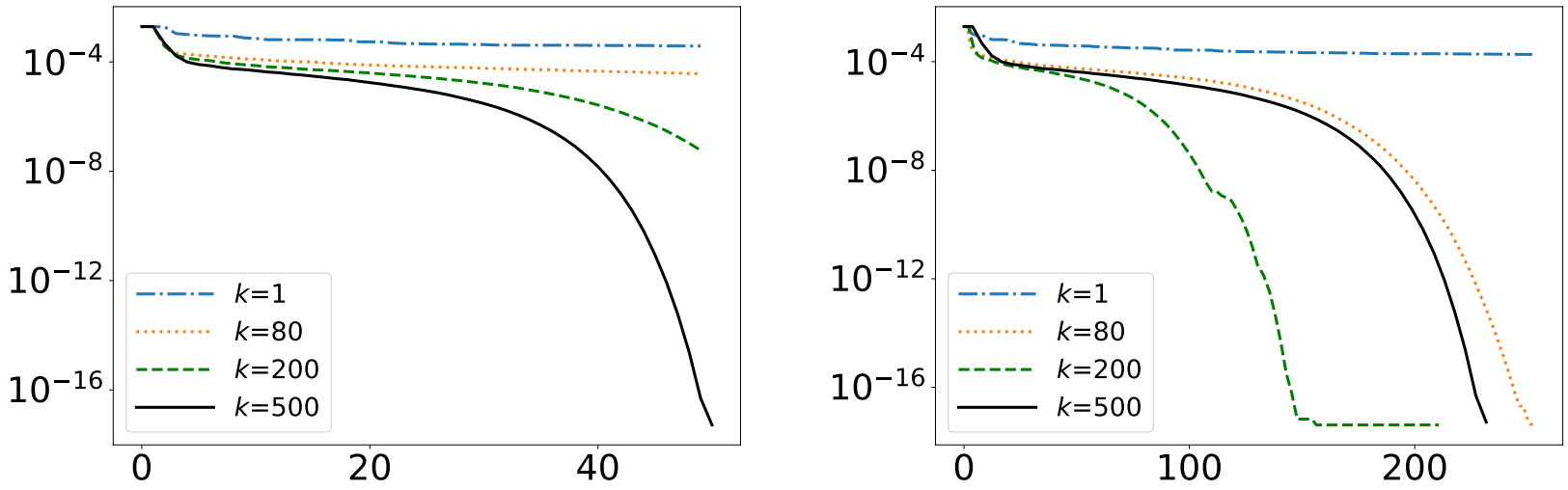


(a) #iteration vs. $\|\nabla f(\mathbf{x}_t)\|$ (b) time (s) vs. $\|\nabla f(\mathbf{x}_t)\|$

The results of the model of regularized logistic regression on "sido0" with $k = 80$, $d = 5000$.

SR-k: Experiments on logistic regression

Convergence behavior of SR-k with different k



(a) #iteration vs. $\|\nabla f(\mathbf{x}_t)\|$ (b) time (s) vs. $\|\nabla f(\mathbf{x}_t)\|$

The results of the model of regularized logistic regression on "sido0" with $k = 80$, $d = 5000$.

SQN: From convex optimization to more

Let's consider the minimax optimization

$$\min_{\mathbf{x}\in\mathbb{R}^{d_x}} \max_{\mathbf{y}\in\mathbb{R}^{d_y}} f(\mathbf{x},\mathbf{y}).$$

We denote $\mathbf{z} = [\mathbf{x}^\top; \mathbf{y}^\top]^\top$ and assume f is μ-strong convex on $\mathbf{x}$ and is μ-strong concave on $\mathbf{y}$, $\nabla f(\cdot)$, $\nabla^2 f(\cdot)$ are lipschitz continuous.
The Hessian of f

$$\nabla^2 f(\mathbf{x},\mathbf{y}) = \begin{bmatrix} \nabla^2_{\mathbf{xx}} f(\mathbf{x},\mathbf{y}) \succeq \mu\mathbf{I} & \nabla^2_{\mathbf{xy}} f(\mathbf{x},\mathbf{y}) \\ \nabla^2_{\mathbf{yx}} f(\mathbf{x},\mathbf{y}) & \nabla^2_{\mathbf{yy}} f(\mathbf{x},\mathbf{y}) \preceq -\mu\mathbf{I} \end{bmatrix}$$

is indefinite!

SQN

Denote $\mathbf{z} = (\mathbf{x}, \mathbf{y})$, $\mathbf{g}_t = \nabla f(\mathbf{x}_t, \mathbf{y}_t)$, $\mathbf{H}_t = \nabla^2 f(\mathbf{x}_t, \mathbf{y}_t)$ and $\hat{\mathbf{H}}_t = \mathbf{H}_t^2$.

We rewrite the Newton update

$$\begin{aligned}
\mathbf{z}_{t+1} =& \mathbf{z}_k - \left(\nabla^2 f(\mathbf{z}_t)\right)^{-1} \nabla f(\mathbf{z}_t) \\
=& \mathbf{z}_k - \left(\nabla^2 f(\mathbf{z}_t)\right)^{-1} \left(\nabla^2 f(\mathbf{z}_t)\right)^{-1} \left(\nabla^2 f(\mathbf{z}_t) \nabla f(\mathbf{z}_t)\right) \\
=& \mathbf{z}_k - \left(\hat{\mathbf{H}}_t^{-1}\right) \mathbf{H}_t \mathbf{g}_t.
\end{aligned}$$

Some good news

- $\hat{\mathbf{H}}_t = \mathbf{H}_t^2 \succeq \mu^2 \mathbf{I}$
- $\hat{\mathbf{H}}_t \mathbf{g}_t$ and $\mathbf{H}_t \mathbf{g}_t$ can be computed efficiently

Approximating $\hat{\mathbf{H}}_t$ leads to quasi-Newton for minimax!

SQN

Algorithm Squared-Quasi-Newton Methods (SQN)

Input: $\mathbf{z}_0 \in \mathbb{R}^d$, $\mathbf{G}_0 \succeq L^2\mathbf{I}$ and $M \geq 0$.

for $k = 0, 1 \ldots$

$\quad \mathbf{z}_{t+1} = \mathbf{z}_t - \mathbf{G}_t^{-1}\mathbf{H}_t\mathbf{g}_t$

$\quad r_t = \|\mathbf{z}_{t+1} - \mathbf{z}_t\|$

$\quad \tilde{\mathbf{G}}_t = (1 + Mr_t)\mathbf{G}_t$

$\quad \mathbf{u}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$\quad \mathbf{G}_{t+1} = \mathrm{SR1}(\tilde{\mathbf{G}}_t, \underbrace{\hat{\mathbf{H}}_{t+1}}_{[\nabla^2 f(\mathbf{z}_{t+1})]^2}, \mathbf{u}_t).$

end for

SQN

Theorem (convergence for SQN)

SQN converges at the following rate in a local region of $\mathbf{x}^*$

$$\lambda_{t+t_0} \leq \left(1 - \frac{1}{d}\right)^{\frac{t(t-1)}{2}} \left(1 - \frac{1}{4\kappa^2}\right)^{t_0} \left(\frac{1}{2}\right)^t \lambda_0$$

with probability $1 - \delta$*, where* $\kappa = L/\mu$*,* $d = d_x + d_y$*,*

$$\lambda_t = \|\nabla f(\mathbf{x}_t, \mathbf{y}_t)\|_2 \quad \textit{and} \quad t_0 = \mathcal{O}\left(\left(d + \kappa^2\right) \ln\left(\frac{d\kappa}{\delta}\right)\right).$$

SQN for nonlinear equations

Let's consider an even more challenge problem

$$\mathbf{F}(\mathbf{z}) = \mathbf{0}, \qquad \mathbf{F} : \mathbb{R}^d \to \mathbb{R}^m$$

Let $\mathbf{J}(\mathbf{z})$ be Jacobian of $\mathbf{F}$ at $\mathbf{z}$. $\mathbf{J}(\mathbf{z}) \in \mathbb{R}^{m \times d}$ is even not square!
We suppose

- there exist $\mathbf{z}^*$ such that $\mathbf{J}(\mathbf{z}^*)$ is non-singular;
- $\mathbf{J}(\cdot)$ is L_2-Lipschitz continuous.

The LM methods

$$\mathbf{z}_{t+1} = \mathbf{z}_t - (\mathbf{J}(\mathbf{z}_t)^\top \mathbf{J}(\mathbf{z}_t) + \lambda_t \mathbf{I})^{-1} \mathbf{J}(\mathbf{z}_t)^\top \mathbf{F}(\mathbf{z}_t).$$

The quasi-Newton methods for nonlinear equations

$$\begin{cases} \mathbf{z}_{t+1} = \mathbf{z}_t - \mathbf{G}_t{}^{-1} \mathbf{J}(\mathbf{z}_t)^\top \mathbf{F}(\mathbf{x}_t) \\ \tilde{\mathbf{G}}_t = (1 + M\|\mathbf{z}_{t+1} - \mathbf{z}_t\|)\mathbf{G}_t \\ \mathbf{G}_{t+1} = \mathrm{SR1}(\tilde{\mathbf{G}}_t, \mathbf{J}(\mathbf{z}_{t+1})^\top \mathbf{J}(\mathbf{z}_{t+1}), \mathbf{U}_t). \end{cases}$$

SQN: Machine Learning Application

- **AUC maximization** aims to find the classifier $\mathbf{w} \in \mathbb{R}^m$ on the training set $\{\mathbf{a}_i, b_i\}_{i=1}^n$ where $\mathbf{a}_i \in \mathbb{R}^m$ and $b_i \in \{+1, -1\}$.

$$\min_{\mathbf{x} \in \mathbb{R}^{m+2}} \max_{y \in \mathbb{R}} f(\mathbf{x}, y) \triangleq \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x}, \mathbf{y}; \mathbf{a}_i, b_i, \lambda),$$

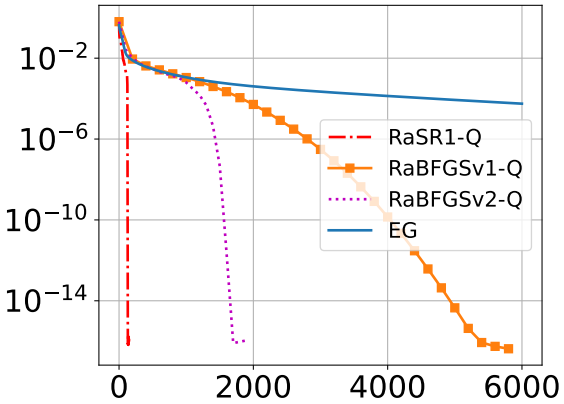
  where $\mathbf{x} = [\mathbf{w}; u; v] \in \mathbb{R}^{m+2}$, $d_x = m + 2$ and $d_y = 1$.

- **Fairness-aware machine learning** aims to make the input variable $c_i \in \mathbb{R}$ unbiased on the training set $\{\mathbf{a}_i, b_i, c_i\}_{i=1}^n$.

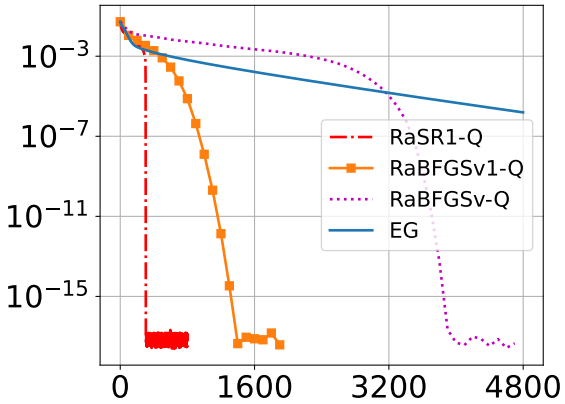
$$\min_{\mathbf{x} \in \mathbb{R}^m} \max_{y \in \mathbb{R}} \frac{1}{n} \sum_{i=1}^n (l_1(b_i \mathbf{a}_i^\top \mathbf{x}) - \beta l_2(c_i y \mathbf{a}_i^\top \mathbf{x})) + \lambda \|\mathbf{x}\|^2 - \gamma y^2,$$

  where $d_x = m$, $d_y = 1$.

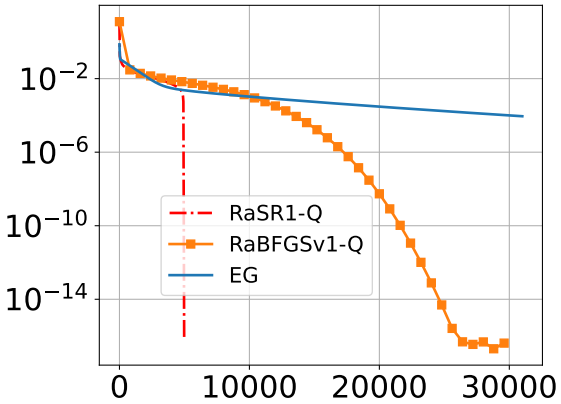
SQN: Experiments on AUC maximization



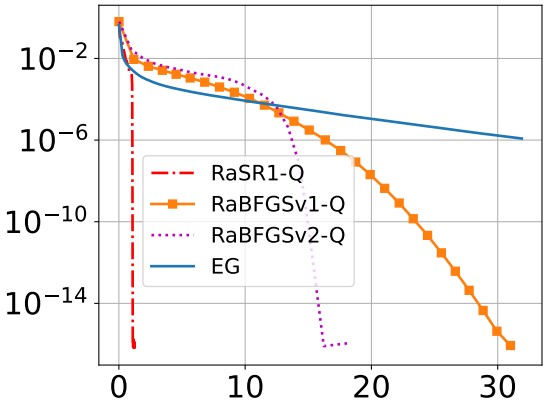
(a) a9a (iteration)



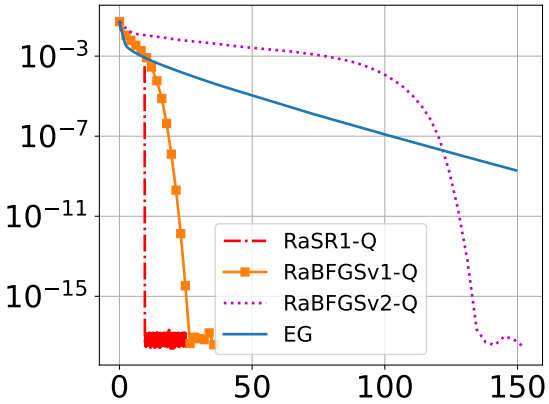
(b) w8a (iteration)



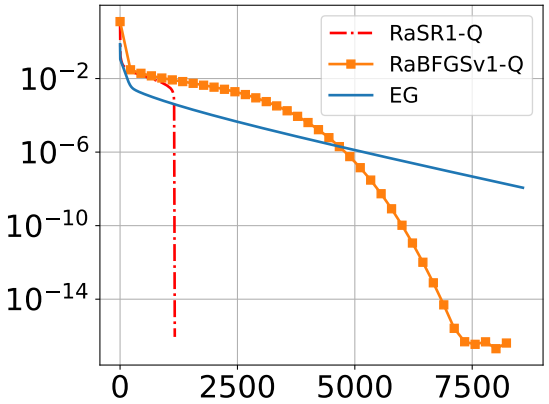
(c) sido0 (iteration)



(d) a9a (time)

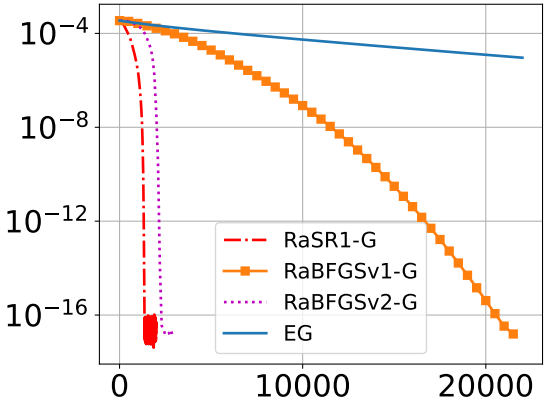


(e) w8a (time)

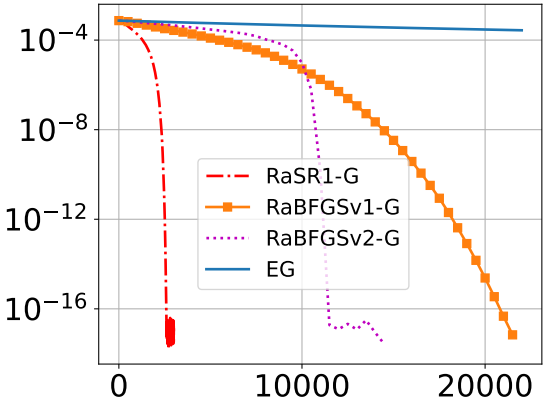


(f) sido0 (time)

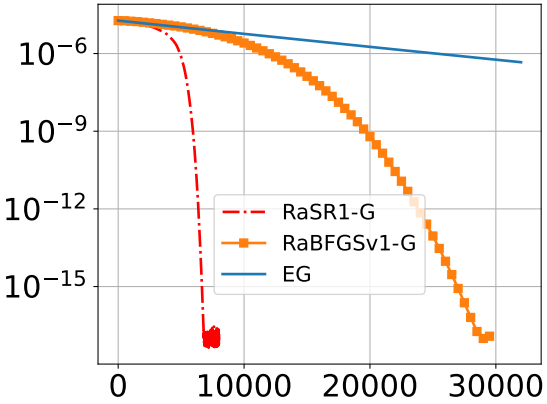
SQN: Experiments on Adversarial Debiasing



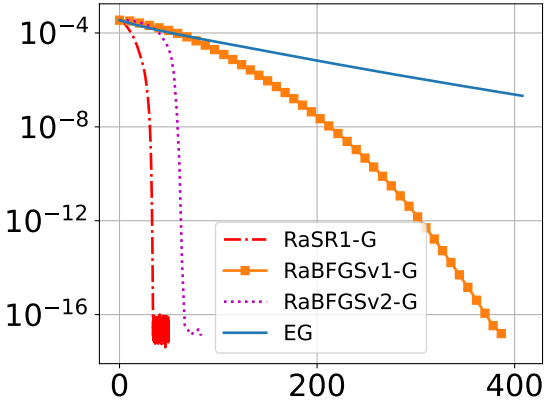
(a) adults (iteration)



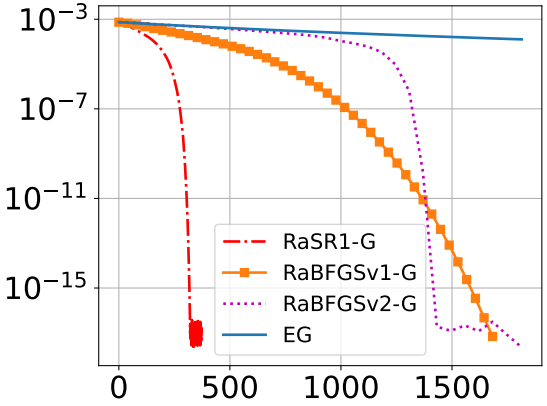
(b) school (iteration)



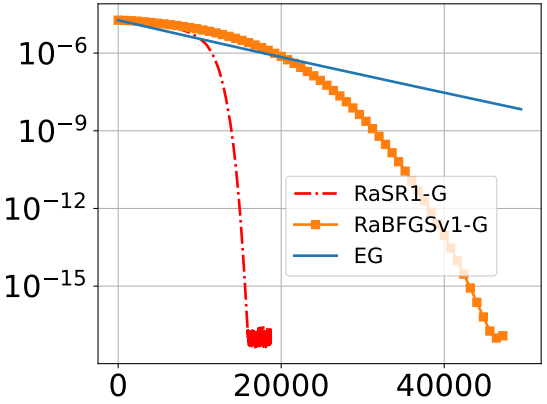
(c) bank (iteration)



(d) adults (time)



(e) school (time)



(f) bank (time)

PQN: Faster methods for minimax optimization

The weakness of squared-quasi-Newton methods:

- It does not differ $\mathbf{x}$ and $\mathbf{y}$.
- The greedy method could be so expensive.
- The square of $\nabla^2 f(\mathbf{x}, \mathbf{y})$ has worse condition number.

There is "unbalanced dimensionality" in real applications.

- For AUC maximization/adversarial debiasing, we have $d_x \gg d_y = 1$.
- Computing the partial Hessian could be very cheap.

PQN

Partial-Quasi-Newton methods construct the Hessian estimator by

$$\nabla^2 f(\mathbf{x}_t, \mathbf{y}_t) = \begin{bmatrix} \nabla^2 f_{\mathbf{xx}}(\mathbf{x}_t, \mathbf{y}_t) & \nabla^2 f_{\mathbf{xy}}(\mathbf{x}_t, \mathbf{y}_t) \\ \nabla^2 f_{\mathbf{yx}}(\mathbf{x}_t, \mathbf{y}_t) & \nabla^2 f_{\mathbf{yy}}(\mathbf{x}_t, \mathbf{y}_t) \end{bmatrix}$$
$$\approx \begin{bmatrix} \mathbf{G}_t & \nabla^2 f_{\mathbf{xy}}(\mathbf{x}_t, \mathbf{y}_t) \\ \nabla^2 f_{\mathbf{yx}}(\mathbf{x}_t, \mathbf{y}_t) & \nabla^2 f_{\mathbf{yy}}(\mathbf{x}_t, \mathbf{y}_t) \end{bmatrix},$$

where $\mathbf{G}_t$ is obtained by Broyden family update.

PQN

Algorithm Partial-Quasi-Newton Methods (PQN)

Input: $\mathbf{G}_{\mathbf{xx},0}$, $\mathbf{z}_0$ and $M \geq 0$
$\mathbf{G}_0^{-1} = \mathrm{Inverse}\big(\mathbf{G}_{\mathbf{xx},0}^{-1}, \mathbf{H}_{\mathbf{xy},0}, \mathbf{H}_{\mathbf{yy},0}\big)$
for $k = 0, 1, \ldots$
    $\mathbf{z}_{t+1} = \mathbf{z}_t - \mathbf{G}_t^{-1}\mathbf{g}(\mathbf{z}_t)$
    $r_t = \langle \mathbf{x}_{t+1} - \mathbf{x}_t, \mathbf{P}(\mathbf{z}_t)(\mathbf{x}_{t+1} - \mathbf{x}_t)\rangle^{1/2} + \|\mathbf{y}_{t+1} - \mathbf{y}_t\|$
    $\tilde{\mathbf{G}}_{\mathbf{xx},t} = (1 + Mr_t)\mathbf{G}_{\mathbf{xx},t}$
    Choose $\mathbf{u}_t$ by greedy/random method
    $\mathbf{G}_{\mathbf{xx},t+1} = \mathrm{SR1}\big(\tilde{\mathbf{G}}_{\mathbf{xx},t}, \mathbf{H}_{\mathbf{xx}}(\mathbf{z}_{t+1}), \mathbf{u}_t\big)$
    $\mathbf{G}_{t+1}^{-1} = \mathrm{Inverse}\big(\mathbf{G}_{\mathbf{xx},k+1}^{-1}, \mathbf{H}_{\mathbf{xy}}(\mathbf{z}_{t+1}), \mathbf{H}_{\mathbf{yy}}(\mathbf{z}_{t+1})\big)$
end for

PQN

For the Optimal First-Order (EG/OGDA) method, we have

$$\rho_{t+t_0} \leq \left(1 - \frac{1}{4\varkappa}\right)^{k+t_0} \rho_0 \quad \text{where} \quad \rho_t = \|\mathbf{z}_t - \mathbf{z}^*\|^2.$$

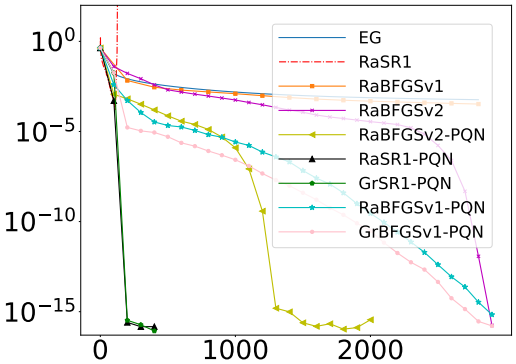
For the Squared-Quasi-Newton method, we have

$$\lambda_{t+t_0} \leq \left(1 - \frac{1}{d}\right)^{\frac{t(t-1)}{2}} \left(1 - \frac{1}{4\varkappa^2}\right)^{t_0} \left(\frac{1}{2}\right)^t \lambda_0 \quad \text{where} \quad \lambda_t = \|\nabla f(\mathbf{z}_t)\|.$$

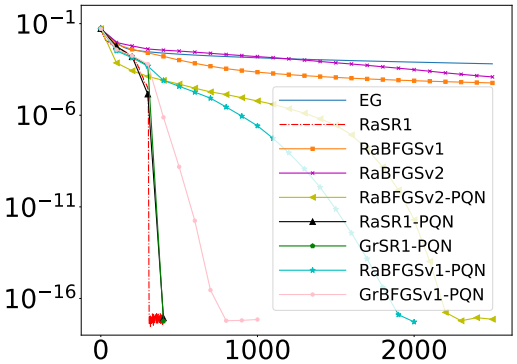
For the Partial-Quasi-Newton method, we have

$$\gamma_{t_0+t} \leq \left(1 - \frac{1}{d}\right)^{\frac{t(t-1)}{2}} \left(\frac{1}{2}\right)^t \left(1 - \frac{1}{4\varkappa}\right)^{t_0} \gamma_0.$$

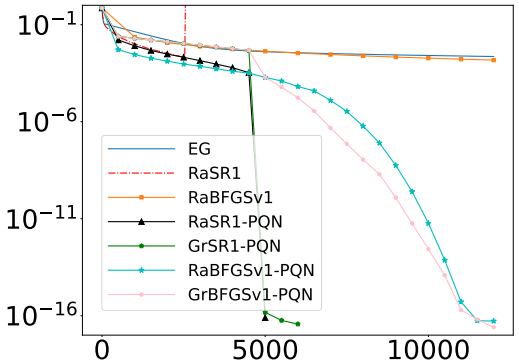
PQN: Experiments on AUC Maximization



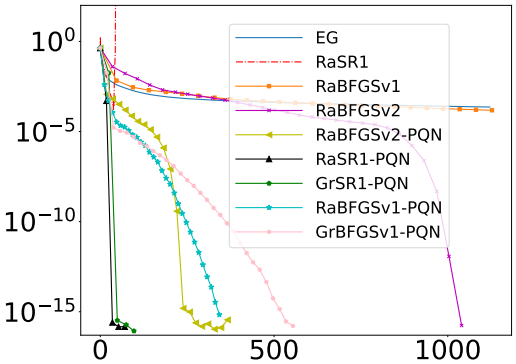
(a) a9a (iteration)



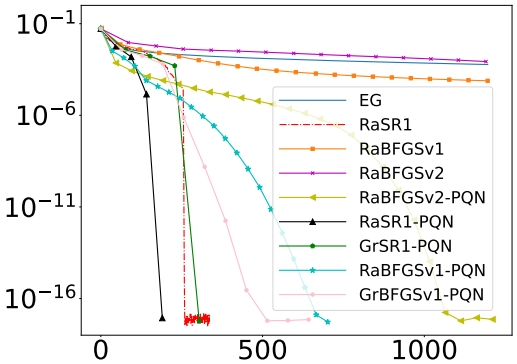
(b) w8a (iteration)



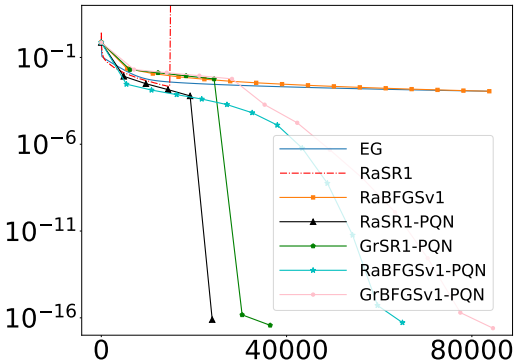
(c) sido0 (iteration)



(d) a9a (time)



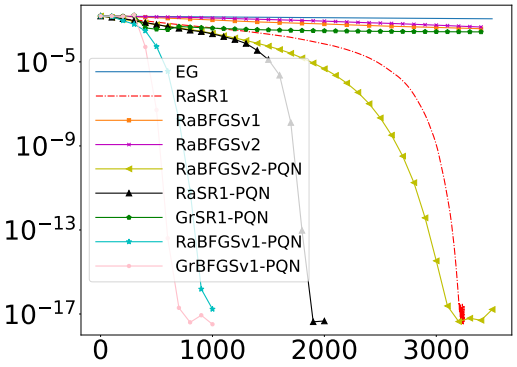
(e) w8a (time)



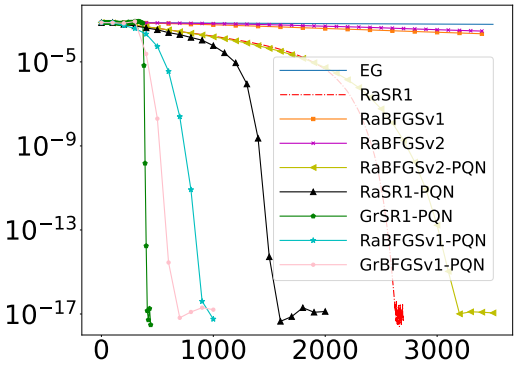
(f) sido0 (time)

Results on different datasets for AUC model

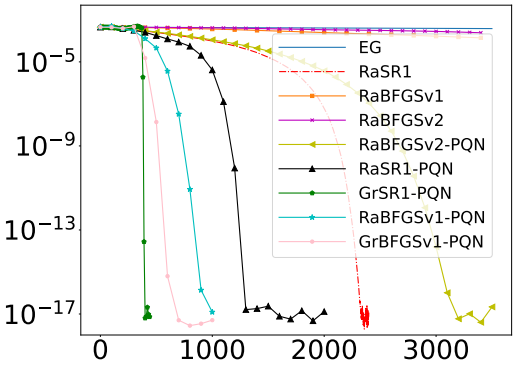
PQN: Experiments on Adversarial Debiasing



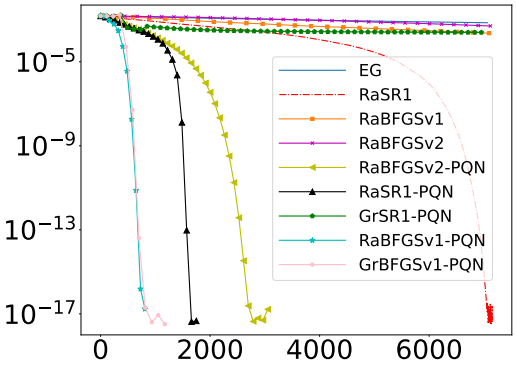
(a) $T = 20$ (iteration)



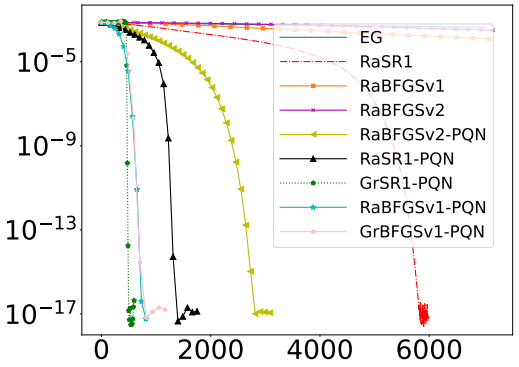
(b) $T = 200$ (iteration)



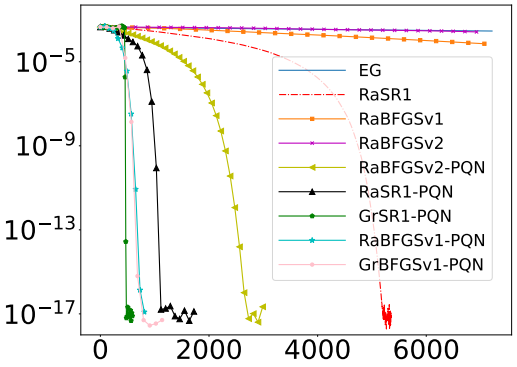
(c) $T = 2000$ (iteration)



(d) $T = 20$ (time)



(e) $T = 200$ (time)



(f) $T = 2000$ (time)

Results on "school" dataset with different iteration numbers of warm-up

Outline

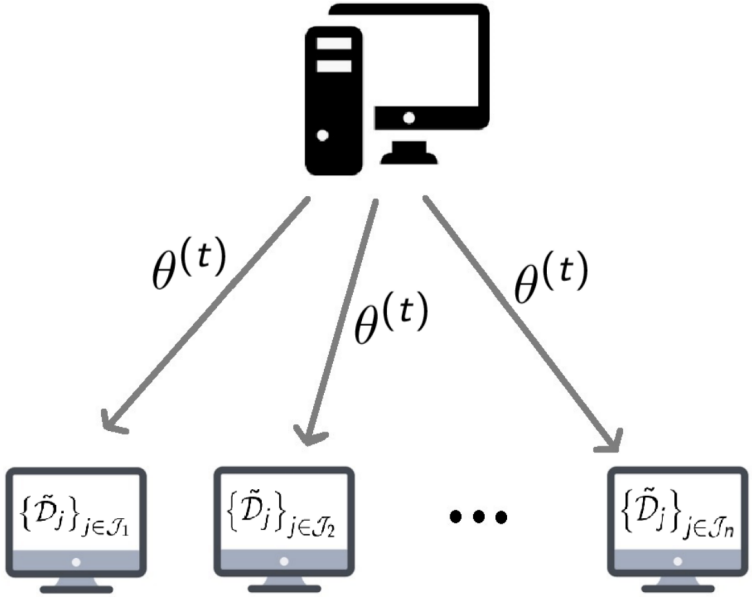
1. Why Second-Order Methods?
2. Computation Efficient Methods for Single Agent Optimization
   - SR-k: Bridging the quasi-Newton and the Newton
   - SQN: From convex optimization to more
   - PQN: Faster convergence by exploiting special structure
3. Comunication Efficient Methods for Distributed Optimization
4. Conclusion

Distributed Newton

Now, let's consier the multi agent optimization (**centralized** distributed optimization).

$$\min_{\mathbf{x}\in\mathbb{R}^d} f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x}), \tag{5}$$

where n is the numbers of the devices, each f_i is twice-differentiable.



First-order methods for distributed optimization

Distributed gradient descent is very popular for many tasks.

- The client i: calculate $\nabla f_i(\mathbf{x}_t)$
- The server: $\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{1}{L_1} \underbrace{\left(\frac{1}{n} \sum_{i=1}^{n} \nabla f_i(\mathbf{x}_t) \right)}_{\text{received from each client}} = \mathbf{x}_t - \frac{1}{L_1} \nabla f(\mathbf{x}_t)$

Communication cost at each iteration: $\mathcal{O}(d)$ $(\nabla f_i(\mathbf{x}_t), \mathbf{x}_{t+1})$

Hardness for second-order for distributed optimization

Consider distributed (cubic-regularized) Newton method.

- The client i: calculate $\nabla f_i(\mathbf{x}_t)$, $\nabla^2 f_i(\mathbf{x}_t)$
- The server:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \left(\underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla^2 f_i(\mathbf{x}_t)}_{\text{received from each client}} \right)^{-1} \left(\underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x}_t)}_{\text{received from each client}} \right)$$
$$= \mathbf{x}_t - [\nabla^2 f(\mathbf{x}_t)]^{-1} \nabla f(\mathbf{x}_t)$$

Communication cost at each iteration $\mathcal{O}(d^2)$ ($\nabla^2 f_i(\mathbf{x}_t)$, $\nabla f_i(x_t)$ and $\mathbf{x}_{t+1}$)
Communicating the Hessian is bad for distributed optimization!

Designing communication efficient second-order methods

Communication Efficient: At each iteration, the communication cost between the server and the client is $\mathcal{O}(d)$.

Local Descent (GIANT/Local-Cubic):

- The client i: $\mathbf{x}_{i,t+1} = \mathbf{x}_t - \nabla^2 f_i(\mathbf{x}_t)^{-1} \nabla f(\mathbf{x}_t)$
- The server: $\mathbf{x}_{t+1} = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_{i,t+1}}_{\text{received from each client}} \neq \mathbf{x}_t - [\nabla^2 f(\mathbf{x}_t)]^{-1} \nabla f(\mathbf{x}_t)$

(neither $\mathcal{O}(\epsilon^{-3/2})$ for non-convex nor superlinear for strongly-convex).

Designing communication efficient second-order methods

Solve quadratic sub-problems: The (Cubic)-Newton step can be reformulated to

$$\min_{\mathbf{y}} f(\mathbf{x}_t) + \langle \nabla f(\mathbf{x}_t), \mathbf{y} - \mathbf{x}_t \rangle + \frac{1}{2}(\mathbf{y} - \mathbf{x}_t)^\top \nabla^2 f(\mathbf{x}_t)(\mathbf{y} - \mathbf{x}_t), \quad (6)$$

The gradient of equation (6) at $\mathbf{y}$ is

$$\nabla f(\mathbf{x}_t) + \nabla^2 f(\mathbf{x}_t)(\mathbf{y} - \mathbf{x}_t) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x}_t) + \left(\frac{1}{n}\sum_{i=1}^{n} \nabla^2 f_i(\mathbf{x}_t)\right)(\mathbf{y} - \mathbf{x}_t)$$

which can be communicated in $\mathcal{O}(d)$.

Solving the quadratic sub-problem at each iteration will increase the total cost and leads to complicated implementation.

Designing communication efficient second-order methods

Compressed Newton methods: Doing compression before sending the second-order information to the server. These methods enjoy local superlinear rate, however

- it requires constructing local Hessian estimator at each client which raise the space complexity to $\mathcal{O}(d^2)$
- No theoretical results for non-convex case.

Designing communication efficient second-order methods

Method	Communication	Space	NC	SC
Distributed-GD	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-2})$	linear
Distributed-Newton	$\mathcal{O}(d^2)$	$\mathcal{O}(d^2)$	$\mathcal{O}(\epsilon^{-3/2})$	superlinear
GIANT/DANE	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-2})$	linear
Distributed sub Newton	$\mathcal{O}(d)$	$\mathcal{O}(d)$	unknown	linear
Compressed Newton	$\mathcal{O}(d)$	$\mathcal{O}(d^2)$	unknown	superlinear

Distributed Newton methods

We address the raised problems of designing the communication effcient distirbuted Newton methods in the following paper

- **Chengchang Liu**, Lesi Chen, Luo Luo and John C.S. Lui. Communication Efficient Distributed Newton Method with Fast Convergence Rates. *KDD*, 2023

C2EDEN

We developed **C**ommunication and **C**omputation **E**fficient **D**istribut**E**d **N**ewton Method (C2EDEN) which fixed the previous gaps.

Method	Communication	Space	NC	SC
Distributed-GD	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-2})$	linear
Distributed-Newton	$\mathcal{O}(d^2)$	$\mathcal{O}(d^2)$	$\mathcal{O}(\epsilon^{-3/2})$	superlinear
GIANT/DANE	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-2})$	linear
Distributed sub Newton	$\mathcal{O}(d)$	$\mathcal{O}(d)$	unknown	linear
Compressed Newton	$\mathcal{O}(d)$	$\mathcal{O}(d^2)$	unknown	superlinear
C2EDEN	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(\epsilon^{-3/2})$	**superlinear**

C2EDEN: Observations

- Communicate a full Hessian is difficult, but we can atomize the Hessian

$$\nabla^2 f_i(\tilde{\mathbf{x}}) = \begin{bmatrix} | & | & & | \\ \nabla^2 f_i(\tilde{\mathbf{x}})\mathbf{e}_1 & \nabla^2 f_i(\tilde{\mathbf{x}})\mathbf{e}_2 & \cdots & \nabla^2 f_i(\tilde{\mathbf{x}})\mathbf{e}_d \\ | & | & & | \end{bmatrix}$$

  where communicating $\nabla^2 f_i(\tilde{\mathbf{x}})\mathbf{e}_p \in \mathbb{R}^d$ (a Hessian-vector) requires only $\mathcal{O}(d)$.
- We can communicate a full Hessian in d iterations ($\frac{d^2}{d} = d$)
- We can communicate the full Hessian in d iterations while updating the model!

C2EDEN: algorithm

At iteration $t = kd, kd+1 \cdots, (k+1)d-1$,

- The client: compute $\nabla f_i(\mathbf{x}_t)$, $\nabla^2 f_i(\mathbf{x}_{kd})\mathbf{e}_{t-kd+1}$
- The server:

$$\mathbf{H}(:, t-kd+1) = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla^2 f_i(\mathbf{x}_{kd})\mathbf{e}_{t-kd+1}}_{\text{from current iteration}}$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \left[\underbrace{\nabla^2 f(\mathbf{x}_{(k-1)d})}_{\text{from iteration } (k-1)d \text{ to } kd-1}\right]^{-1} \left(\underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x}_t)}_{\text{from current iteration}}\right)$$

Communication cost at each iteration: $\mathcal{O}(d)$ $(\nabla^2 f_i(\tilde{\mathbf{x}})\mathbf{e}_p,\ \nabla f_i(\mathbf{x}_t),\ \mathbf{x}_{t+1})$

C2EDEN: comparison

At iteration $t = kd, kd+1 \cdots, (k+1)d-1$,

C2EDEN

- The client: compute $\nabla f_i(\mathbf{x}_t)$, $\nabla^2 f_i(\mathbf{x}_{kd})\mathbf{e}_{t-kd+1}$
- The server:

$$\mathbf{H}(:, t-kd+1) = \frac{1}{n}\sum_{i=1}^{n} \nabla^2 f_i(\mathbf{x}_{kd})\mathbf{e}_{t-kd+1}$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - [\nabla^2 f(\mathbf{x}_{(k-1)d})]^{-1}\left(\frac{1}{n}\sum_{i=1}^{n}\nabla f_i(\mathbf{x}_t)\right)$$

Distributed-Newton

- The client: compute $\nabla f_i(\mathbf{x}_t)$, $\nabla^2 f_i(\mathbf{x}_t)$
- The server:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - [\nabla^2 f(\mathbf{x}_t)]^{-1}\left(\frac{1}{n}\sum_{i=1}^{n}\nabla f_i(\mathbf{x}_t)\right)$$

C2EDEN: convergence results

The theorems are based on [Nesterov and Polyak 2006], [Doikov et.al 2022]

Theorem (Nonconvex case)

Suppose the Hessian of the object function f is L_2-Lipschitz continuous, then with no more than $\mathcal{O}(\sqrt{dL_2}\epsilon^{-3/2})$, C2EDEN finds a stationary point $\mathbf{x}_i$ satisfied

$$\|\nabla f(\mathbf{x}_i)\| \leq \epsilon \quad \textit{and} \quad \lambda_{\min}(\nabla^2 f(\mathbf{x}_i)) \geq -\sqrt{dL_2\epsilon}$$

Theorem (Strongly-convex case)

Suppose the Hessian of the object function f is L_2-Lipschitz continuous and f is μ-strongly convex, then C2EDEN converges at a superlinear rates locally, i.e.,

$$\lim_{k\to\infty} \frac{\|\nabla f(\mathbf{x}_{t+1})\|}{\|\nabla f(\mathbf{x}_t)\|} = 0$$

C2EDEN: empirical study

We validate C2EDEN on regularized logistic regression.

$$f_i(\mathbf{x}) = \frac{1}{m_i} \sum_{j=1}^{m_i} \ln \left(1 + \exp\left(-b_{ij} \mathbf{x}^\top \mathbf{a}_{ij}\right)\right) + \lambda R(\mathbf{x}), \tag{7}$$

For the nonconvex case, we set

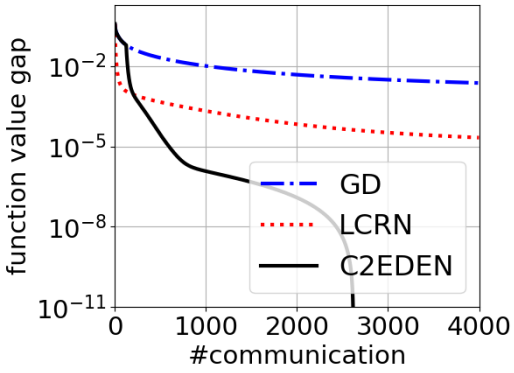
$$R(\mathbf{x}) = \sum_{p=1}^{d} \frac{x_{(p)}^2}{1 + x_{(p)}^2}$$

For the strongly convex case, we set

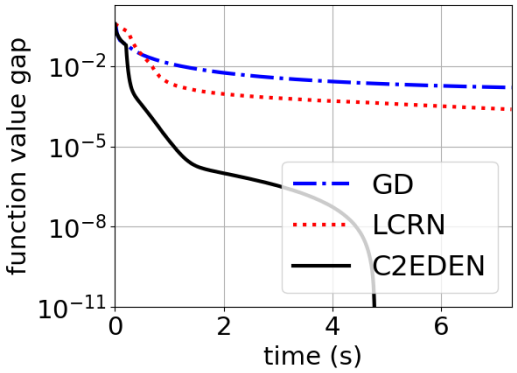
$$R(\mathbf{x}) = \|\mathbf{x}\|_2^2$$

C2EDEN: empirical study

Nonconvex case:



(a) #communication vs. gap

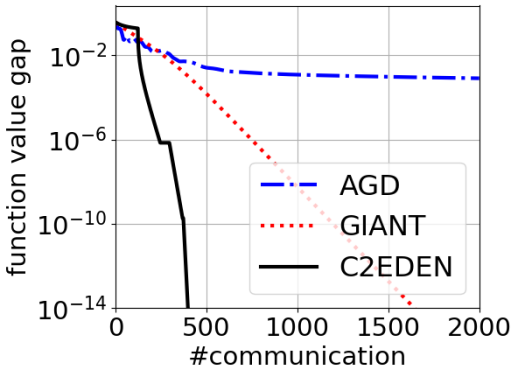


(b) time (s) vs. gap

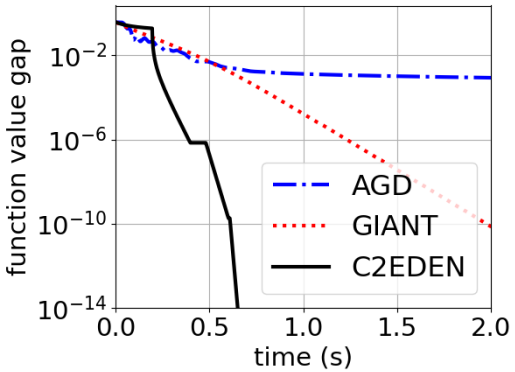
The results of the model of nonconvex regularized logistic regression on "a9a".

C2EDEN: empirical study

Strongly-convex case:



(a) #communication vs. gap



(b) time (s) vs. gap

The results of the model of l_2-regularized logistic regression on "a9a".

Outline

1. Why Second-Order Methods?
2. Computation Efficient Methods for Single Agent Optimization
   - SR-k: Bridging the quasi-Newton and the Newton
   - SQN: From convex optimization to more
   - PQN: Faster convergence by exploiting special structure
3. Comunication Efficient Methods for Distributed Optimization
4. Conclusion

Coclusion

My work focus on designing second order optimization methods with computation (and communication) efficiency:

- For the single-agent optimization, computing the Hessian and its inverse is expensive, we develop new quasi-Newton methods.
  - SR-k builds a bridge between quasi-Newton methods and the Newton methods [arXiv 2023].
  - SQN extends the theory from convex optimization to the minimax optimization and the nonlinear equations [NeurIPS 2022]
  - PQN exploits special structure of minimax and provides faster convergence rates [KDD 2022]
- For the multi-agent optimization, communicating Hessian is hard, we develop C2DEDEN which achieves both of the communication and computation efficiency [KDD 2023].

Coclusion

- Faster convergence guarantee (which can not be achieved by any variants of first-order methods) **fast**
- Relatively cheap computation (and communication) cost (which are almost the same cost as the first-order methods) **efficient**